Lab 5: Static Code Analysis

Student: Ruthik **SRN:** PES1UG25CS837

Important: This is a prepared example and draft, not a completed report from the instructor-assigned Python program. The actual assigned script and genuine before/after tool reports are needed to claim the lab is complete.

Objective

Use Pylint, Bandit, and Flake8 to identify code-quality, security, and style issues in a Python program, then fix and verify them.

Issue table (example only)

Issue/tool	Type	Description	Approach
Input validation	Robustness	Path may be empty	Check input and show a message
File handling	Reliability	File reads can fail	Catch expected file/encoding errors
Error reporting	Maintainability	Need useful context	Raise a clear error and preserve cause
Code organization	Style	Separate functions and execution	Use main() and __main__ guard

Reflection answers (draft)

1. What is static code analysis? It checks source code for possible defects, security issues, and style problems without executing the program.

2. What does Pylint do? It reports code-quality issues, possible mistakes, and maintainability concerns.

3. What does Bandit do? It checks Python code for common security risks.

4. What does Flake8 do? It reports style and formatting problems, including many PEP 8 violations.

5. Why rerun tools after fixing code? To verify that addressed issues no longer appear and to check whether new issues were introduced.

6. What did you learn? Static analysis can identify issues early and encourage consistent, maintainable, and safer code. Adjust this answer to describe the work actually completed.

Commands for analysis

```
python -m pip install pylint bandit flake8
pylint static_code_analysis.py > pylint_report_after.txt
bandit -r static_code_analysis.py > bandit_report_after.txt
flake8 static_code_analysis.py > flake8_report_after.txt
```

The lab handout also requires before-and-after reports and three commits. Generate the before reports from the real assigned starter program before editing; do not invent report results.